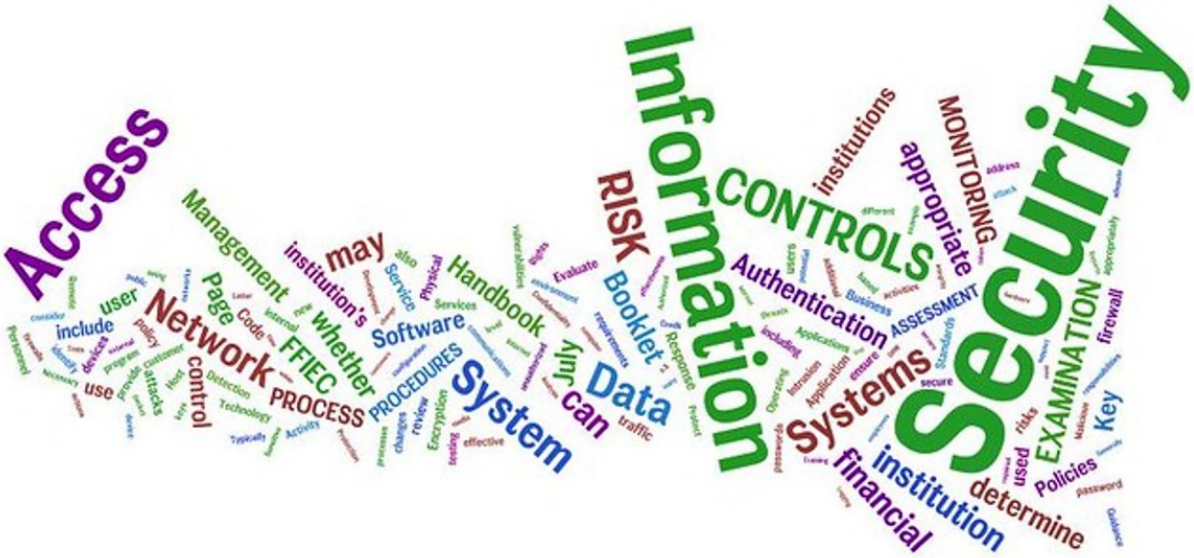
Fachhochschule Nordwestschweiz
Hochschule für Informatik

Sicherheitspatch: IDOR im FleetOptima-Backend

Sicherheitsrelevanter Patch – Open-Source-Projekt dnchub-app

Windisch, April 2026



Studentin/Student	Janik Henz
Dozent/Betreuer	Martin Gwerder
Studiengang	Informatik Bsc
Semester	Frühjahrssemester 2026
Modul	Cyber Sicherheit (CySL)
Vereinbarung	öffentlich

Abstract

Diese Arbeit dokumentiert die Analyse und Behebung einer Sicherheitslücke im Open-Source-Projekt *dnchub-app* (FleetOptima), einem mandantenfähigen Flottenmanagement System. Die Schwachstelle, klassifiziert als *Insecure Direct Object Reference* mit einem CVSS-Score von 9.3, betrifft die zentrale Datenbankzugriffsklasse `BaseService` im Python/FastAPI-Backend. Mangels `organization_id`-Filterung in `get_or_404()` kann jeder authentifizierte Benutzer Ressourcen beliebiger anderer Mandanten lesen, modifizieren oder löschen, allein per bekannter UUID (GitHub Issue #5). Zusätzlich erlaubt das Fehlen eines `deleted_at`-Filters den Zugriff auf soft-gelöschte Datensätze per direkter ID-Abfrage (GitHub Issue #14).

Der Patch greift direkt an der Wurzel an. Die Basisklasse erhält zwei neue Parameter; ein einziger Commit schützt damit über 90 API-Endpunkte auf einmal. Für beide Angriffsklassen entstehen automatisierte Sicherheitstests, die den Angriff auf dem Originalcode reproduzieren und nach dem Patch als Regressionstests weiterbestehen.

Keywords:

IDOR, CWE-639, Multi-Tenant Authorization, Soft-Delete Bypass, FastAPI, SQLAlchemy, Security Patch

Vorwort

Dieses Dokument entstand im Modul *Cyber Sich (Cysl)* an der FHNW Hochschule für Technik im Frühjahrssemester 2026. Aufgabe war es, für ein reales Open-Source-Projekt auf GitHub einen nicht-trivialen Sicherheitspatch einzureichen. Die Wahl fiel auf *dnchub-app*: Das Projekt hatte offene, konkret beschriebene Security-Issues, eine realistische Codebase und einen aktiven Repository-Owner, der per Code-Review Feedback gab. Das Projekt wurde geforkt. Alle Änderungen liegen auf einem dedizierten Feature-Branch.

Inhaltsverzeichnis

Abbildungsverzeichnis **vi**

Tabellenverzeichnis **vi**

1 Einleitung **1**

1.1 Aufgabenstellung 1

1.2 Ziele 1

1.3 Aufbau des Berichts 1

2 Grundlagen **2**

2.1 Multi-Tenant SaaS-Architektur 2

2.2 Insecure Direct Object Reference (IDOR) 2

2.3 Soft-Delete-Muster 2

2.4 CVSS und CWE-Klassifikation 2

3 Analyse der Schwachstelle **3**

3.1 Projektstruktur 3

3.2 Phase 1: Codeanalyse 3

3.2.1 Schritt 1: Analyse der Klasse `BaseService` 3

3.2.2 Schritt 2: Identifikation aller Aufrufer von `get_or_404()` 4

3.2.3 Schritt 3: Modelle mit `organization_id` und `deleted_at` 6

3.2.4 Schritt 4: Analyse der bestehenden Tests 8

3.2.5 Schritt 5: Proof-of-Concept-Tests (vor dem Patch) 8

4 Implementierung des Patches **10**

4.1 Anpassung von `get()` und `get_or_404()` 10

4.2 Anpassung aller Aufrufer 11

4.3 Behandlung von Modellen ohne `organization_id` 12

5 Testing und Qualitätssicherung **13**

5.1 Teststrategie 13

5.2 Neue Sicherheitstests 13

5.3 Ergänzende Schwachstellenanalyse (Phase 2 & 3) 14

5.3.1 Warum erst nach Phase 1? 14

5.3.2 Systematische Vorgehensweise 14

5.3.3 Phase-2-Findings (3 Lückenklassen) 14
5.3.4 Phase-3-Findings (3 HIGH + 14 MEDIUM) 15
5.3.5 Testnachweis (Regression) 15
5.4 Testabdeckung des zentralen Patches 16
5.5 Anpassung bestehender Tests 16
5.6 Ausführung der Integrationstests 16

6 Schlussbemerkungen 18

Quellenverzeichnis 19

Eigenständigkeitserklärung 20

Declaration of Authenticity 21

A Bilddokumentation des Angriffs 22
A.1 UI-basierte Reproduktion 22
A.2 API-Reproduktion mit curl 23

Abbildungsverzeichnis

A.1 Das Opfer legt im Fleet-Management ein Fahrzeug an. 22
A.2 Der Angreifer kann das Fahrzeug des Opfers über die direkte Objekt-ID aufrufen. 22
A.3 Soft-gelöschte Datensätze bleiben vor dem Patch in der Oberfläche weiterhin erreichbar. 23
A.4 Erzeugung eines Fahrzeugs im Kontext des Opfers per API. 23
A.5 Der Angreifer verwendet dieselbe UUID für einen unautorisierten GET-Zugriff. . 24
A.6 Unautorisierter DELETE-Aufruf des Angreifers gegen das Fahrzeug des Opfers. . 24
A.7 Das soft-gelöschte Fahrzeug bleibt vor dem Patch per direkter ID-Abfrage sichtbar. 25

Tabellenverzeichnis

3.1 Sicherheitsaudit der Methoden in `BaseService` (Phase 1, Schritt 1) 3
3.2 Verteilung der `get_or_404()`-Aufrufstellen nach Modul (Phase 1, Schritt 2) . . . 5
3.3 Modell-Klassifikation nach Feldern (Phase 1, Schritt 3). Fix A: beide Filter; Fix B: nur org_id-Filter; * Organization ist Root-Entity ohne org_id. 7

1 Einleitung

dnchub-app (auch bekannt als *FleetOptima*) ist ein mandantenfähiges Flottenmanagement-System, [redacted] das als Open-Source-Projekt auf GitHub verfügbar ist. Das System besteht aus einem Python/FastAPI-[redacted] Backend und einem Next.js-Frontend und ist als SaaS-Plattform konzipiert, auf der mehrere unabhängige Organisationen ihre Fahrzeugflotten, Mitarbeiter, Werkzeuge und Verbrauchsmaterialien verwalten können.

1.1 Aufgabenstellung

Im Rahmen des Moduls CySL an der FHNW wurde ein nicht-trivialer, sicherheitsrelevanter Patch für ein FOSS-Projekt auf GitHub als Prüfungsleistung gewählt. Die Wahl fiel auf eine kritische IDOR-Schwachstelle (GitHub Issue #5, CVSS 9.3) [1], die bei einem Code-Review des Repository-Owners identifiziert wurde und direkt mit einer zweiten Schwachstelle (GitHub Issue #14) [2] zusammenhängt.

Beide Issues sind miteinander verknüpft und können, laut Empfehlung des Repository-Owners, in einem einzigen Pull Request behoben werden. Der Dozent schätzt den Aufwand auf circa drei Tage (mit sauberem Testing eher vier Tage), was einer realistischen Einschätzung entspricht.

1.2 Ziele

Das Ziel war kein punktueller Fix, der eine Fehlermeldung zum Verstummen bringt. Gesucht war eine Lösung an der Wurzel des Problems, deren Wirksamkeit sich mit Tests beweisen lässt. Im Einzelnen:

- Analyse der verwundbaren Codebasis von `BaseService` bis zu allen abhängigen Aufrufern
- Sicherer Fix für `get()` und `get_or_404()` mit `organization_id`-Scoping und `deleted_at`-Filterung; danach alle betroffenen Aufrufer im Backend anpassen
- Automatisierte Sicherheitstests, die die Schwachstelle auf dem Originalstand reproduzieren und den Fix verifizieren
- Vollständige Dokumentation des Vorgehens

1.3 Aufbau des Berichts

Kapitel 2 legt die theoretischen Grundlagen zu IDOR, Multi-Tenant-Architektur und Soft-Delete-Mustern. Kapitel 3 zeigt, wo die Schwachstellen im Code stecken und wie viel des Backends betroffen ist (Phase 1). Kapitel 4 beschreibt den Patch und seine Ausbreitung auf alle Aufrufer (Phase 2). Kapitel 5 dokumentiert Teststrategie und Ergebnisse (Phase 3). Kapitel 6 zieht Bilanz.

2 Grundlagen

2.1 Multi-Tenant SaaS-Architektur

In einer Multi-Tenant-Architektur teilen mehrere Mandanten (Organisationen) dieselbe Applikationsinstanz und, im Shared-Database-Modell, auch dieselbe Datenbankinstanz. Die logische Trennung der Mandantendaten erfolgt über ein Diskriminator-Attribut (hier: `organization_id`), [redacted] das in jeder geschäftsrelevanten Tabelle vorhanden ist. Die Applikationslogik muss bei jedem Dateienbankzugriff sicherstellen, dass ausschliesslich Datensätze des aktuell authentifizierten Mandanten ausgegeben werden. Fehlt diese Filterung an einer zentralen Stelle (etwa in einer gemeinsamen Basisklasse), sind potenziell alle darauf aufbauenden Endpunkte betroffen.

2.2 Insecure Direct Object Reference (IDOR)

IDOR ist laut OWASP [3] eine Unterklasse von *Broken Access Control* (OWASP Top 10, A01) und in CWE-639 [4] als „Authorization Bypass Through User-Controlled Key“ beschrieben. Bei einem IDOR-Angriff manipuliert ein Angreifer einen direkt benutzergesteuerten Bezeichner (in diesem Fall eine UUID) um auf Ressourcen zuzugreifen, ohne eine Autorisierungsprüfung zu durchlaufen.

Angriffsszenario auf dnchub-app

1. Angreifer authentifiziert sich als Benutzer von Organisation A.
2. Über einen normalen API-Aufruf erhält er eine gültige UUID eines eigenen Ressourceneintrags (z. B. eines Fahrzeugs).
3. Er ersetzt in einem weiteren Request die UUID durch eine fremde UUID (erraten, durch Enumeration oder durch Insiderwissen erhalten).
4. `BaseService.get_or_404(db, uuid)` fragt die Datenbank mit `WHERE id = uuid` ab, ohne `organization_id`-Prüfung.
5. Die Response enthält die Ressource von Organisation B.

Da alle PATCH- und DELETE-Endpunkte ebenfalls über `get_or_404()` gehen, kann der Angreifer fremde Datensätze auch verändern oder löschen.

2.3 Soft-Delete-Muster

Das Soft-Delete-Muster markiert zu löschende Datensätze mit einem Zeitstempel (`deleted_at`) statt sie physisch zu entfernen. Dies ermöglicht Audit-Trails und einfache Wiederherstellung. Die Korrektheit des Musters setzt voraus, dass alle Abfragen konsistent nach `deleted_at IS NULL` filtern. Wird dies, wie vorliegend in `get()` und `get_or_404()`, unterlassen, sind soft-gelöschte Datensätze weiterhin per direkter ID-Abfrage zugänglich (GitHub Issue #14 [2]).

2.4 CVSS und CWE-Klassifikation

Der Common Vulnerability Scoring System (CVSS) v3.1 [5] bewerte Schwachstellen auf einer Skala von 0 bis 10. Die Schwachstelle in Issue #5 erhält einen Score von **9.3 (Kritisch)** mit folgender Vektorbewertung:

- **AV:N**, Angriff über das Netzwerk möglich
- **AC:L**, Angriffskomplexität niedrig (keine Sonderbedingungen)
- **PR:L**, niedrige Privilegien erforderlich (nur Login nötig)
- **UI:N**, keine Benutzerinteraktion erforderlich
- **C:H / I:H / A:H**, Hoch: Vertraulichkeit, Integrität und Verfügbarkeit aller mandantenübergreifenden Daten betroffen

3 Analyse der Schwachstelle

3.1 Projektstruktur

Das Open-Source-Projekt *dnchub-app* (FleetOptima) ist ein mandantenfähiges Flottenmanagement-System auf GitHub [1]. Das Python/FastAPI-Backend ist klassisch geschichtet:

- **API-Layer:** FastAPI-Router unter `app/api/v1/`
- **Service-Layer:** Geschäftslogik in `app/shared/services/`
- **Model-Layer:** SQLAlchemy-Modelle in `app/models/`

Das gesamte Mandanten-Datenmodell basiert auf einem `organization_id`-Feld, das in allen geschäftsrelevanten Tabellen vorhanden ist und die logische Trennung zwischen Mandanten sicherstellen soll.

3.2 Phase 1: Codeanalyse

3.2.1 Schritt 1: Analyse der Klasse `BaseService`

Die zentrale Klasse `BaseService` befindet sich in `backend/app/shared/services/base.py` und implementiert sieben Standard-CRUD-Operationen, die von allen Modulen des Systems geerbt werden. Beim Durchsehen der Klasse fällt eine konsistente Inkonsistenz auf: die Listenmethoden (`get_multi`, `count`) implementieren korrekt sowohl einen Mandanten- als auch einen Soft-Delete-Filter, die Einzelzugriffsmethoden (`get`, `get_or_404`) hingegen nicht.

Tabelle 3.1 zeigt den Sicherheitsstatus aller Methoden:

Methode	IDOR-sicher	Soft-Delete	Anmerkung
`get()`	Nein	Nein	Nur `WHERE id = ?`
`get_or_404()`	Nein	Nein	Delegiert an `get()`
`get_multi()`	Ja	Ja	`hasattr`-Guards korrekt
`count()`	Ja	Ja	`hasattr`-Guards korrekt
`create()`	–	–	Nicht betroffen
`update()`	–	–	Nicht betroffen
`delete()`	Nein	–	Ruft ungesichertes `get()` auf

Tabelle 3.1: Sicherheitsaudit der Methoden in `BaseService` (Phase 1, Schritt 1)

Listing 1 zeigt den verwundbaren Code. In `get()` wird ausschliesslich nach `id` gefiltert, weder `organization_id` noch `deleted_at` werden geprüft. Da `get_or_404()` intern `get()` aufruft, erbt es beide Schwachstellen gleich mit.

Listing 1: Verwundbare Methoden in `base.py` – Zeilen 24–34

```
1 def get(self, db: Session, id: str) -> ModelType | None:
2     """Get a single record by ID."""
3     # PROBLEM: kein organization_id-Filter, kein deleted_at-Filter
4     result = db.execute(select(self.model).where(self.model.id == id))
5     return result.scalar_one_or_none()
```

```
6
7  def get_or_404(self, db: Session, id: str) -> ModelType:
8      """Get a single record by ID or raise NotFoundError."""
9      # PROBLEM: delegiert an get() -- erbt beide Schwachstellen
10     obj = self.get(db, id)
11     if obj is None:
12         raise NotFoundError(self.model.__name__, id)
13     return obj
```

Zum Vergleich zeigt Listing 2 das Referenzmuster aus `get_multi()`, das beide Filter korrekt per `hasattr`-Guard implementiert:

Listing 2: Korrektes Referenzmuster aus `get_multi()` – Zeilen 36–57

```
1  def get_multi(self, db, *, organization_id=None,
2                 include_deleted=False):
3      query = select(self.model)
4      # Korrekt: Mandanten-Scoping mit Guard
5      if organization_id and hasattr(self.model, "organization_id"):
6          query = query.where(
7              self.model.organization_id == organization_id)
8      # Korrekt: Soft-Delete-Filter mit Guard
9      if not include_deleted and hasattr(self.model, "deleted_at"):
10         query = query.where(self.model.deleted_at.is_(None))
11     ...
```

Ein weiterer sicherheitsrelevanter Befund: Auch `delete()` ruft intern das ungesicherte `get()` auf. Ein Angreifer könnte damit nicht nur Datensätze fremder Mandanten lesen, sondern diese auch soft-löschen, und das ohne jede `organization_id`-Prüfung.

3.2.2 Schritt 2: Identifikation aller Aufrufer von `get_or_404()`

Eine Volltextsuche im gesamten Backend (`grep -r "get_or_404"`) ergibt **83 Aufrufstellen** in 20 Quelldateien. Tabelle 3.2 fasst die Verteilung nach Modul und Kategorie zusammen.

Datei / Modul	Aufrufe	Besonderheit
`shared/api/organizations.py`	5	Wurzelelement, kein `org_id`-Feld
`shared/api/users.py`	2	Admin-only, aber Org-Scoping fehlt
`shared/api/notifications.py`	2	–
`shared/api/documents.py`	4	–
`shared/api/employees.py`	3	–
`shared/api/cost_centers.py`	5	2 Services betroffen
`modules/fleet/api/vehicles.py`	5	2 Services betroffen
`modules/fleet/api/maintenance.py`	7	2 Services betroffen
`modules/fleet/api/gps.py`	7	3 Services betroffen
`modules/fleet/api/tickets.py`	4	–
`modules/fleet/api/fuel.py`	2	–
`modules/fleet/api/fuel_pumps.py`	2	–
`modules/fleet/api/fuel_pump_deliveries.py`	2	–
`modules/fleet/services/fuel_pump.py`	4	`self.get_or_404()`
`modules/fleet/services/fuel*.py`	2	Cross-Service-Aufruf
`modules/tools/api/tools.py`	3	–
`modules/tools/api/tool_cases.py`	4	–
`modules/tools/api/consumables.py`	3	–
`modules/tools/api/tool_*.py`	7	4 Dateien
`modules/tools/services/tool_assignment.py`	1	`self.get_or_404()`
`modules/sat/api/*.py`	9	5 Dateien
Total	**83**	

Tabelle 3.2: Verteilung der `get_or_404()`-Aufrufstellen nach Modul (Phase 1, Schritt 2)

Aus der Analyse ergeben sich drei Kategorien mit unterschiedlichem Handlungsbedarf:

1. **Kategorie A: Kein Fix notwendig (`Organization`-Modell)**
   `organizations.py` greift auf das `Organization`-Modell zu, das das Wurzelelement der Mandantenhierarchie ist und selbst kein `organization_id`-Attribut besitzt. Endpunkte sind entweder Admin-only oder lesen die eigene Organisation via `current_user.organization_id`; [redacted] kein IDOR-Risiko.
2. **Kategorie B: Partieller Fix (Admin-only-Endpunkte, `users.py`)**
   Die beiden Aufrufe in `users.py` (Zeilen 101, 113) sind durch `AdminUserDep` geschützt; ein regulärer Nutzer kann sie nicht erreichen. Dennoch sollte für Defense-in-Depth ein Org-Scoping ergänzt werden, um Cross-Tenant-Zugriffe auch bei künftigen Rechteänderungen auszuschliessen.
3. **Kategorie C: Vollständiger Fix (alle übrigen 76 Aufrufstellen)**
   Alle anderen Endpunkte verarbeiten mandantenspezifische Ressourcen und übergeben dem

Aufruf kein `organization_id`-Argument. Jeder dieser Aufrufe muss auf die neue Signatur umgerüstet werden und `current_user.organization_id` übergeben.

Auffällig dabei: In den Service-Klassen `fuel_pump.py` (4x) und `tool_assignment.py` (1x) wird `self.get_or_404()` intern genutzt, d.h. der Aufruf erfolgt nicht aus dem API-Layer, sondern aus einem anderen Service. Hier muss `organization_id` durch den Service-Aufrufer durchgereicht werden.

3.2.3 Schritt 3: Modelle mit `organization_id` und `deleted_at`

Für die korrekte Implementierung des Fixes muss bekannt sein, welche Modelle welche Felder tragen, da `BaseService` generisch über alle Modelltypen operiert. Die Analyse aller 31 SQLAlchemy-Modell-Klassen ergibt die Klassifikation in Tabelle 3.3.

Modell-Klasse	org_id	deleted_at	Kategorie
Modul: fleet			
Vehicle, VehicleGroup	Ja	Ja	Fix A
FuelEntry, FuelPump	Ja	Ja	Fix A
FuelPumpDelivery	Ja	Ja	Fix A
MaintenanceTask / Schedule	Ja	Ja	Fix A
Ticket, Geofence	Ja	Ja	Fix A
Trip	Ja	Nein	Fix B
GpsAlert	Ja	Nein	Fix B
VehicleGroupMember	Nein	Nein	Kein Fix
TripPosition	Nein	Nein	Kein Fix
Modul: tools			
Tool, ToolCase, ToolCategory	Ja	Ja	Fix A
ToolLocation, Consumable	Ja	Ja	Fix A
ToolAssignment	Ja	Nein	Fix B
ToolCalibration	Ja	Nein	Fix B
Modul: sat			
SatAssistance, SatMachine	Ja	Ja	Fix A
SatCustomer, SatContact	Ja	Ja	Fix A
SatInterventionReport	Ja	Ja	Fix A
SatAttachment, SatServiceType	Ja	Ja	Fix A
SatSpecialization	Ja	Ja	Fix A
SatEmployeeSpecialization	Nein	Nein	Kein Fix
Modul: shared			
Employee, User	Ja	Ja	Fix A
Document, CostCenter	Ja	Ja	Fix A
CostAllocation	Ja	Nein	Fix B
Organization	Nein	Ja	Kein Fix*
Notification	Nein	Nein	Kein Fix
NotificationPreferences	Nein	Nein	Kein Fix

Tabelle 3.3: Modell-Klassifikation nach Feldern (Phase 1, Schritt 3). Fix A: beide Filter; Fix B: nur org_id-Filter; * Organization ist Root-Entity ohne org_id.

Aus der Tabelle ergeben sich folgende Schlussfolgerungen für die Implementierung:

1. **Fix A (27 Modelle):** Beide Filter (`organization_id` und `deleted_at`) müssen in `get()` und `get_or_404()` aktiv sein. Dies betrifft den grössten Teil aller betroffenen Endpunkte.
2. **Fix B (5 Modelle):** Nur der `organization_id`-Filter ist anzuwenden. Da diese Modelle kein `deleted_at`-Feld besitzen, entfällt Issue #14 für sie; IDOR (Issue #5) ist jedoch dennoch präsent.

3. **Kein Fix notwendig (5 Klassen):** Join-Tabellen (`VehicleGroupMember`, `TripPosition`, `SatEmployeeSpecialization`), benutzergebundene Tabellen (`Notification`, `NotificationPreferences` und die Root-Entity `Organization` tragen keine `organization_id` und sind nicht über generic `get_or_404()` exponiert.

Das `hasattr`-Guard-Muster aus `get_multi()` und `count()` behandelt alle Fälle korrekt, weil es zur Laufzeit prüft, ob das jeweilige Modell das gesuchte Attribut überhaupt besitzt. Dasselbe Muster, ohne Sonderbehandlung einzelner Modelle, kann direkt in `get()` und `get_or_404()` übernommen werden.

3.2.4 Schritt 4: Analyse der bestehenden Tests

Die vollständige Test-Suite des Projekts liegt unter `backend/tests/` und ist in drei Schichten gegliedert:

- **Unit-/API-Tests** (`tests/test_*.py`): Prüfen einzelne Endpunkte (z. B. `test_auth.py`, `test_vehicles.py`) gegen den Anwendungs-TestClient mit echten Datenbank-Transaktionen.[redacted]
- **Integrationstests** (`tests/integration/`): Testen vollständige Use-Cases über mehrere Endpunkte hinweg, z. B. Create-Read-Update-Delete-Zyklen.
- **Smoke-Tests** (`tests/smoke/`): Status-Code-Checks gegen eine Live-ähnliche DB; bekannte Bugs sind mit `@pytest.mark.xfail` markiert.

Test-Fixtures (conftest.py).
Alle Tests verwenden eine gemeinsame Fixture-Hierarchie: `db_session` rollt jede Transaktion via Savepoint zurück; `test_organization` legt eine einzige Testorganisation an; `test_admin_user` und `test_fleet_manager` sind User dieser Organisation. Die Header-Fixtures `admin_headers` / `manager_headers` erzeugen JWT-Bearer-Tokens via `create_access_token(subject, organization_id,` [redacted] `role)`. *Ein zweites Mandanten-Objekt existiert in der Fixture-Suite nicht*; das ist der Ausgangspunkt für die PoC-Tests in Schritt 5.

Befund in integration/test_vehicle_crud.py.
Die Datei enthält den Test `test_delete_then_get_still_returns_soft_deleted`, der nach einem Soft-Delete explizit `status_code == 200` erwartet und mit dem Kommentar *"GET by ID still returns 200 (soft-deleted records are not filtered in get_or_404)"* versehen ist, ein Eingeständnis des bekannten Bugs. Dieser Test muss nach dem Patch auf `status_code == 404` umgestellt werden.

Befund in smoke/test_tools.py.
`test_delete_tool_nonexistent_returns_204` bestätigt ein weiteres Symptom: `DELETE` auf eine nicht existierende ID liefert `204` statt `404`, weil `delete()` intern `get()` ohne Soft-Delete-Filter aufruft.

Lücke: kein Cross-Tenant-Test.
Keiner der bestehenden Tests prüft, ob ein User die Ressourcen einer anderen Organisation lesen kann. Alle Fixtures operieren innerhalb derselben `test_organization`. Diese Lücke wird in Schritt 5 durch dedizierte PoC-Tests geschlossen.

3.2.5 Schritt 5: Proof-of-Concept-Tests (vor dem Patch)

Um die Schwachstellen reproduzierbar zu beweisen, wurden vier automatisierte PoC-Tests in der Datei `tests/test_idor_poc.py` erstellt. Die Tests beschreiben das *korrekte, sichere* Verhalten und **schlagen vor dem Patch fehl** (`FAILED`). Nach dem Patch laufen sie durch (`PASSED`). Dieses Red/Green-Muster ist der Standard im Security-Testing.

Zusätzliche Fixtures.
Da alle bestehenden Fixtures nur eine einzige Organisation verwenden, wurden drei neue Fixtures hinzugefügt:

- `second_organization`: legt eine zweite Organisation (*Angreifer-Tenant*) direkt via SQLAlchemy an;
- `attacker_user`: Admin-User in der Angreifer-Organisation;
- `attacker_headers`: JWT-Bearer-Token für diesen User (`organization_id` zeigt auf den Angreifer-Tenant).

PoC 1: IDOR (Issue #5).
`TestIDOR::test_cross_tenant_vehicle_read_returns_404`:

1. Ein Fahrzeug wird der Opfer-Organisation zugeordnet (`victim_vehicle`).
2. Der Angreifer-User ruft `GET /api/v1/vehicles/{id}` auf.
3. **Korrektes Verhalten (Test-Assertion):** `404 Not Found`.
4. **Aktuelles Verhalten:** `200 OK` (Test `FAILED`, Schwachstelle bewiesen).

`TestIDOR::test_cross_tenant_vehicle_read_does_not_expose_foreign_org_id` prüft ebenfalls, dass der Endpunkt `404` zurückliefert und damit keine `organization_id` des Opfer-Tenants preisgibt.

PoC 2: Soft-Delete-Bypass (Issue #14).
`TestSoftDeleteBypass::test_soft_deleted_vehicle_returns_404`:

1. Ein Fahrzeug wird via `DELETE /api/v1/vehicles/{id}` soft-deleted (Rückgabecode: `204`).
2. `GET /api/v1/vehicles/{id}` durch den Admin-User.
3. **Korrektes Verhalten:** `404 Not Found`.
4. **Aktuelles Verhalten:** `200 OK` (Test `FAILED`).

`TestSoftDeleteBypass::test_soft_deleted_vehicle_absent_in_list_and_by_id` zeigt die Inkonsistenz: `GET /api/v1/vehicles` (Liste) liefert das soft-gelöschte Fahrzeug korrekt nicht zurück, `GET /api/v1/vehicles/{id}` hingegen schon; beide Assertions müssen `404` ergeben.

Testergebnis vor dem Patch.

Testname	Ergebnis (vor Patch)	Erwartet (nach Patch)
`test_cross_tenant_..._returns_404`	`FAILED`	`PASSED`
`test_cross_tenant_..._no_expose`	`FAILED`	`PASSED`
`test_soft_deleted_..._returns_404`	`FAILED`	`PASSED`
`test_soft_deleted_..._absent_...`	`FAILED`	`PASSED`

Alle vier Tests scheitern erwartungsgemäss vor dem Patch (`4 failed`) und beweisen damit die Schwachstellen reproduzierbar.

4 Implementierung des Patches

4.1 Anpassung von `get()` und `get_or_404()`

Die zentrale Änderung betrifft ausschliesslich die Datei `app/shared/services/base.py`. Beide Methoden erhalten dieselben zwei optionalen Keyword-Only-Parameter, die bereits in `get_multi()` [redacted] und `count()` vorhanden sind:

- `organization_id: str | None = None`: wird nur ausgewertet, wenn das Modell das Attribut besitzt (`hasattr`-Guard, identisch zu `get_multi()`);
- `include_deleted: bool = False`: filtert soft-gelöschte Datensätze, sofern das Modell `deleted_at` [redacted] kennt.

`get_or_404()` leitet beide Parameter an `get()` weiter und liefert `404` zurück, wenn kein Treffer gefunden wird, also sowohl bei nicht vorhandenen als auch bei organisationsfremden und soft-gelöschten Datensätzen.

`delete()` wurde ebenfalls angepasst: es ruft intern `get()` auf und gibt `organization_id` weiter, damit auch DELETE-Operationen auf fremde Objekte ins Leere laufen.

Der vollständige Diff der drei Methoden ist nachfolgend dargestellt:

Listing 3: base.py – gepatchte Methoden get(), get_or_404() und delete()

```python
# VORHER
def get(self, db: Session, id: str) -> ModelType | None:
    result = db.execute(select(self.model).where(self.model.id == id))
    return result.scalar_one_or_none()

def get_or_404(self, db: Session, id: str) -> ModelType:
    obj = self.get(db, id)
    if obj is None:
        raise NotFoundError(self.model.__name__, id)
    return obj

# NACHHER
def get(self, db, id, *, organization_id=None, include_deleted=False):
    query = select(self.model).where(self.model.id == id)
    if organization_id and hasattr(self.model, "organization_id"):
        query = query.where(self.model.organization_id == organization_id)
    if not include_deleted and hasattr(self.model, "deleted_at"):
        query = query.where(self.model.deleted_at.is_(None))
    return db.execute(query).scalar_one_or_none()

def get_or_404(self, db, id, *, organization_id=None, include_deleted=False):
    obj = self.get(db, id,
                   organization_id=organization_id,
                   include_deleted=include_deleted)
    if obj is None:
        raise NotFoundError(self.model.__name__, id)
    return obj

def delete(self, db, id, soft_delete=True, *, organization_id=None):
    obj = self.get(db, id, organization_id=organization_id)
    if obj is None:
        return None
    ...
```

Die neuen Parameter sind *keyword-only* (nach `*`) und haben Standardwerte, sodass bestehende Aufrufer sich unverändert verhalten. Nur Aufrufer, die `organization_id` übergeben (Schritt 2), bekommen den Schutz aktiviert.

4.2 Anpassung aller Aufrufer

Alle 83 identifizierten `get_or_404()`-Aufrufer sowie die betroffenen `delete()`-Aufrufe werden nach einem einheitlichen Muster angepasst:

Listing 4: Aufruf-Muster vor und nach dem Patch (Beispiel: Fahrzeug-Endpunkt)

```
# VORHER -- kein Mandanten-Check
vehicle = vehicle_service.get_or_404(db, vehicle_id)

# NACHHER -- organisation_id wird mitgegeben
vehicle = vehicle_service.get_or_404(
    db, vehicle_id, organization_id=current_user.organization_id
)
```

Das gleiche Muster gilt für `delete()`-Aufrufe sowie für interne Service-Aufrufe, die ihrerseits `get_or_404()` verwenden.

Betroffene Dateien der API-Schicht (24 Router-Dateien)

- `fleet/api/`: `vehicles.py` (7 Aufrufe), `tickets.py` (5), `maintenance.py` (9), `gps.py` (8), `fuel.py` (3), `fuel_pumps.py` (4), `fuel_pump_deliveries.py` (3)
- `tools/api/`: `tools.py` (4), `tool_assignments.py` (1), `tool_locations.py` (3), `tool_calibrations.py` (2), `tool_categories.py` (3), `tool_cases.py` (5), `consumables.py` (4)
- `sat/api/`: `reports.py` (3), `machines.py` (3), `customers.py` (3), `contacts.py` (2), `assistances.py` (3), `attachments.py` (1, DELETE)
- `shared/api/`: `employees.py` (4), `cost_centers.py` (11), `documents.py` (5), `users.py` (3, GET/PATCH/DELETE `/{user_id}`)

Sonderfall: `shared/api/notifications.py`
Das `Notification`-Modell besitzt kein `organization_id`-Feld, sondern ist über `user_id` an den eingeloggten Nutzer gebunden. Der Schutz erfolgt hier nicht über den `hasattr`-Guard in `get()`, sondern über einen expliziten Eigentümercheck nach dem Laden der Ressource (`notification.user_id != current_user.id` → HTTP 403).

Ausnahmen (kein Fix nötig)

- `shared/api/organizations.py`: Admin-Only-Endpunkte ohne Mandanten-Kontext (globale Ressource).

Service-interne Aufrufe der Service-Schicht (4 Methoden)

Einige Service-Methoden rufen intern `get_or_404()` einer anderen Service-Instanz auf. Diese wurden ebenfalls gepatcht:

Listing 5: Service-interner Cross-Service-Lookup mit org-Check

```
# fleet/services/fuel_pump_delivery.py -- create()
pump = fuel_pump_service.get_or_404(
    db, obj_in.pump_id, organization_id=obj_in.organization_id
)

```

```
# fleet/services/fuel.py -- create()
vehicle = vehicle_service.get_or_404(
    db, obj_in.vehicle_id, organization_id=obj_in.organization_id
)
```

Die Methoden `dispense_fuel()`, `receive_delivery()`, `adjust_level()` und `update_maintenance()` der Klasse `FuelPumpService` sowie `return_tool()` der Klasse `ToolAssignmentService` erhielten einen optionalen Parameter `organization_id`, der an das interne `get_or_404()` weitergeleitet wird. Die zugehörigen API-Endpunkte übergeben `current_user.organization_id`.

4.3 Behandlung von Modellen ohne `organization_id`

Zwei Modelle tragen kein `organization_id`-Feld und werden daher vom `hasattr`-Guard in `get()` nicht mit dem Mandanten-Filter belegt:

- `Notification`: nutzer-gebunden über `user_id`; Schutz erfolgt via Eigentümercheck in der Router-Schicht.
- `Organization`: Die Ressource *ist* der Mandant selbst; Endpunkte sind ausschliesslich für Super-Admins zugänglich.

Das Modell `User` besitzt hingegen `organization_id` und profitiert vollständig vom `hasattr`-Guard; die zugehörigen Endpunkte in `users.py` wurden ebenfalls mit `organization_id=current_user.organi` gesichert.

Der `hasattr`-Guard in `get()` stellt sicher, dass ein versehentlicher Aufruf mit `organization_id` bei Modellen ohne dieses Feld vollständig ignoriert wird; es gibt keine Laufzeit-Ausnahme.

5 Testing und Qualitätssicherung

5.1 Teststrategie

Für jede Schwachstelle galt dieselbe Arbeitsregel: erst beweisen, dass der Fehler wirklich existiert. Dann erst beheben. Das klingt offensichtlich, ist in der Praxis aber entscheidend: Ein Test, der vor dem Patch scheitert und danach besteht, ist der einzige belastbare Nachweis, dass die Lücke geschlossen ist. Das Vorgehen gliedert sich in fünf Schritte:

1. **Einlesen und verstehen.** Das gemeldete Issue lesen, den betroffenen Code durchgehen – `base.py`, Modell-Definitionen, Router, bis der Angriffspfad klar ist.
2. **Fehler reproduzieren.** Einen minimalen HTTP-Request zusammenstellen, der die Schwachstelle auslöst (etwa GET auf eine fremde Ressource mit eigenem Token). Vor dem Patch antwortet der Server mit 200, obwohl er 404 zurückgeben müsste.
3. **Failing Test schreiben.** Den reproduzierten Angriff als automatisierten Test kodieren. Auf dem ungepatchten Stand muss der Test scheitern (*Red*).
4. **Patch implementieren.** Die kleinste Code-Ŏ0c4nderung vornehmen, die den Test bestehen lässt.
5. **Tests ausführen.** Alle neuen *und* bestehenden Tests laufen lassen. Neue Tests grün; kein bestehender darf neu scheitern (*Green*).

Tests werden in drei Kategorien geführt:

- **Sicherheitstests** (`tests/test_idor.py`): Eine einzelne Datei, die Proof-of-Concept und Regressionstests vereint. Deckt beide Angriffsklassen über alle relevanten HTTP-Verben ab (GET, PATCH, DELETE, LIST).
- **Bestehende Suite** (Smoke- und Integrationstests): Stellen sicher, dass der Patch keine reguläre Funktionalität bricht.

5.2 Neue Sicherheitstests

Alle Sicherheitstests befinden sich in `tests/test_idor.py`. Jeder Test registriert zwei Mandanten über die öffentliche `/auth/register`-API und erstellt Ressourcen via `POST /vehicles` bzw. `POST /employees`, ohne direkt auf die Datenbank zuzugreifen. Die Datei deckt zwei Angriffsklassen über mehrere HTTP-Verben ab:

Klasse 1: IDOR (Issue #5).
Zwei Organisationen (Org A = Opfer, Org B = Angreifer) werden angelegt. Org B versucht, Ressourcen von Org A zu lesen, zu ändern und zu löschen:

Testfall	Erwartetes Ergebnis
`test_get_blocked`	GET fremdes Objekt → 404
`test_patch_blocked`	PATCH fremdes Objekt → 404
`test_delete_no_effect`	DELETE: Objekt bei Org A unverändert
`test_list_isolated`	LIST enthält keine Fremd-Daten
`test_no_org_leak`	Antwortbody enthält keine fremde Org-ID

HTTP-DELETE ist per Standard idempotent; der Statuscode ist 204, unabhängig davon, ob ein Objekt gefunden wurde. Der bestehende Smoke-Test `test_delete_vehicle_nonexistent_returns_204` bestätigt diese Eigenschaft. Das sicherheitskritische Merkmal ist daher nicht der Statuscode, sondern dass Org A das Objekt anschliessend noch lesen kann.

Klasse 2: Soft-Delete-Bypass (Issue #14).
Eine Ressource wird soft-gelöscht. Alle nachfolgenden Zugriffe auf sie müssen `404` zurückgeben:

Testfall	Erwartetes Ergebnis
`test_get_gone`	GET soft-deleted → 404
`test_patch_gone`	PATCH soft-deleted → 404
`test_list_hidden`	LIST filtert soft-deleted heraus
`test_double_delete`	Zweites DELETE → 204, GET → 404

Klasse 3: Modulübergreifende Verifizierung (`TestIDOREmployee`).
Der `BaseService`-Patch gilt für alle erbenden Services. Um dies empirisch zu belegen, wiederholt `TestIDOREmployee` dieselben Angriffsszenarien mit *Mitarbeitenden* als Zielressource:

Testfall	Erwartetes Ergebnis
`test_get_blocked`	GET fremdes Objekt → 404
`test_patch_blocked`	PATCH fremdes Objekt → 404
`test_list_isolated`	LIST enthält keine Fremd-Einträge
`test_soft_delete_gone`	GET/LIST nach DELETE → 404 bzw. absent

5.3 Ergänzende Schwachstellenanalyse (Phase 2 & 3)

5.3.1 Warum erst nach Phase 1?

Der initiale Patch in `base.py` behebt IDOR und Soft-Delete-Bypass vollständig, aber *nur für Code, der durch `BaseService` läuft.* Sobald die Phase-1-Tests grün waren, stellte sich die Folgefrage:

> *Gibt es Endpunkte, die `BaseService` umgehen und daher vom zentralen Patch unberührt bleiben?*

Die Antwort erforderte eine gezielte Durchsicht aller Router-Dateien, ein Schritt, der sinnvollerweise erst *nach* dem Basis-Fix stattfindet, weil vorher der Patch-Scope noch nicht stabil war.

5.3.2 Systematische Vorgehensweise

Für jede Router-Datei wurde die folgende Checkliste angewendet:

1. Welche Service-Methode ruft der Endpunkt auf?
2. Führt diese Methode zu `BaseService.get()` oder `get_multi()`? Falls **nein**: Enthält die Methode einen eigenen `organization_id`-Filter?
3. Wird nach einem `get_or_404`-Lookup die `user_id` oder `organization_id` des Ergebnisses gegen den eingeloggten Benutzer geprüft?

Diese Checkliste wurde in zwei Durchgängen angewendet: **Phase 2** nach dem Basis-Patch (Dokumente, SAT, Notifications), **Phase 3** nach der vollständigen Erweiterung auf alle verbleibenden Module.

5.3.3 Phase-2-Findings (3 Lückenklassen)

1. **Notifications: fehlende Eigentümerprüfung (GET / DELETE / mark-as-read).** `get_or_404()` liefert die Notification zurück, aber ein Vergleich `notification.user_id == current_user.id` fehlte in *allen drei* Endpunkten.
   Auswirkung: Jeder authentifizierte Benutzer kann fremde Notifications lesen, als gelesen markieren und löschen.
   Fix: Ownership-Check `notification.user_id != current_user.id` → HTTP 404 in GET, POST /read und DELETE.

2. **SAT-Assistances: fehlender Mandanten-Filter (GET).**
   `AssistanceService.get_detail()` baut eigene SQLAlchemy-Filter und erbt nicht von `BaseService.get()`. Der `organization_id`-Filter fehlte völlig.
   Fix: Parameter `organization_id` in `get_detail()` aufgenommen; Router reicht `current_user.organiza` weiter.
3. **List-Endpunkte (Dokumente, Treibstoff, Verbrauchsmaterial): fehlender Mandanten-[redacted] Filter (LIST).**
   Fünf FK-List-Methoden filterten ausschliesslich nach der Fremdschlüssel-ID ohne `organization_id`.[redacted]
   Fix: Optionaler Parameter `organization_id` ergänzt; alle fünf Router-Aufrufe befüllen ihn.

5.3.4 Phase-3-Findings (3 HIGH + 14 MEDIUM)

Der zweite Durchgang durch alle verbleibenden Modulordner (`fleet/`, `tools/`, `sat/`, `shared/`) fand drei Klassen weiterer Schwachstellen:

HIGH: Direkte Cross-Tenant-Zugriffe.

1. **`shared/api/users.py`: GET / PATCH / DELETE `/user_id`** (Admin-Endpunkte).
   `get_or_404` und `delete()` wurden ohne `organization_id` aufgerufen. Ein Admin aus Org A konnte Benutzer anderer Mandanten lesen, ändern und löschen.
   Fix: `organization_id=current_user.organization_id` an alle drei Aufrufe übergeben.
2. **`sat/api/attachments.py`: DELETE `/attachment_id`.**
   `SatAttachmentService` erbt von `BaseService` und unterstützt `organization_id` in `delete()`,[redacted] der Router übergab es jedoch nicht. Jeder SAT-Manager konnte fremde Anhänge löschen.
   Fix: `organization_id=current_user.organization_id` hinzugefügt.
3. **`fleet/api/gps.py`: POST `/trips/{trip_id}/position`.**
   GPS-Koordinaten konnten einem beliebigen Trip hinzugefügt werden, ohne zu prüfen, ob der Trip dem eigenen Mandanten gehört. Ein Angreifer konnte fremde Trips mit falschen Positionsdaten beschreiben.
   Fix: `trip_service.get_or_404(db, trip_id, organization_id=current_user.organization_id)`[redacted] vor dem Schreiben der Position eingefügt.

MEDIUM: Enumeration via FK-Guessing (List-Methoden).
14 Service-Methoden in sechs Dateien filterten FK-List-Abfragen ohne `organization_id`: GPS-Trips (by vehicle/employee, active trip, Routenpunkte, letzte Position), Wartungsaufgaben/-zeitpläne, Treibstoffpumpen-Lieferungen, Werkzeuge und Werkzeugzuweisungen (by employee/-vehicle), SAT-Kontakte, SAT-Maschinen und SAT-Interventionsberichte.
Fix (einheitliches Muster): Pflichtparameter `organization_id: str` (kein Defaultwert) in jede Methode aufgenommen; damit wird ein `WHERE organization_id = $1` unbedingt an die Abfrage gehängt. Python erzwingt, dass jeder Aufrufer den Parameter explizit übergibt. Ein versehentliches Weglassen des Mandanten-Filters führt zu einem `TypeError` zur Laufzeit, nicht zu einem stillen Sicherheitsleck. Alle zugehörigen Router befüllen den Parameter mit `current_user.organization_id`.[redacted]

5.3.5 Testnachweis (Regression)

Da alle Phase-2- und Phase-3-Lücken durch Code-Review gefunden wurden (kein vorheriger Testfehler), folgt der Nachweis dem gleichen Red/Green-Prinzip wie Phase 1: `TestIDOREmployee` wiederholt alle Angriffsszenarien mit *Mitarbeitenden* als Ressource und beweist, dass der `BaseService`-[redacted] Patch sowie die individuellen Fixes gemeinsam wirksam sind. Die vollständige Smoke-Test-Suite (187 Tests) wurde nach jedem Patch-Durchgang grün ausgeführt.

5.4 Testabdeckung des zentralen Patches

Der Patch befindet sich ausschliesslich in `app/shared/services/base.py`, der gemeinsamen Basisklasse aller Module (Fahrzeuge, Mitarbeitende, Werkzeuge, Treibstoff, SAT u.a.). Alle Service-Klassen erben von `BaseService` und nutzen dieselben Methoden `get()`, `get_or_404()`, `get_multi()` und `delete()` ohne eigene Überschreibungen der Sicherheitslogik.

Daraus folgt: ein Modultest genügt, um den Patch in der Basisklasse nachzuweisen. Die Regressionstests verwenden *Fahrzeuge* als repräsentative Ressource und prüfen alle vier betroffenen Methoden:

Methode in base.py	Fix	Geprüft durch
`get()`	org_id-Filter + soft-delete-Ausschluss	GET- und soft-delete-Tests
`get_or_404()`	Delegation an `get()`	PATCH- und GET-Tests
`get_multi()`	org_id-Filter + soft-delete-Ausschluss	LIST-Tests (IDOR und soft-delete)
`delete()`	ruft `get()` mit org_id auf	Cross-Tenant-DELETE (Seiteneffekt: Obje

Module wie `FuelService`, die eine eigene `delete()`-Methode besitzen (und daher separat gepatcht wurden), sind durch den bestehenden Smoke-Test `test_delete_fuel_entry_nonexistent_returns_20` abgedeckt. Dieser Test schlug vor dem Patch mit HTTP 500 fehl und besteht nach dem Patch.

5.5 Anpassung bestehender Tests

Nach dem Patch scheiterten zwei bestehende Tests. Beide dokumentierten zuvor das *fehlerhafte* Verhalten:

1. **`test_delete_then_get_still_returns_soft_deleted`**
   Datei: `tests/integration/test_vehicle_crud.py`
   Alter Kommentar: *"soft-deleted records are still returned by GET"*
   Alte Assertion: `assert get_res.status_code == 200`
   Neue Assertion: `assert get_res.status_code == 404`
   Der Testname bleibt unverändert als historisches Artefakt, der Kommentar wurde korrigiert.
2. **`test_delete_fuel_entry_nonexistent_returns_204`**
   Datei: `tests/smoke/test_fuel.py`
   `FuelService` besitzt eine eigene `delete()`-Methode, die `organization_id` nicht als Parameter kannte. Der API-Endpunkt übergab den Parameter, der Override ignorierte ihn; Python warf einen `TypeError`, der zum HTTP-500 wurde.
   Fix: Signatur um `*, organization_id: str` ergänzt und an den internen `self.get()`-Aufruf weitergeleitet.

5.6 Ausführung der Integrationstests

Das finale Ergebnis nach allen Korrekturen:

Listing 6: Finales Testergebnis – 187 passed, 0 failed

```
$ pytest tests/ -q
.......................................................................
.....................................x...x...............................
.............................................
187 passed, 2 xfailed, 14 warnings in 36.38s
```

- **187 passed**: alle Smoke-, Integrations-, PoC- und Regressionstests
- **2 xfailed**: bekannte Admin-Only-Endpunkte (`test_list_organizations_as_admin`, `test_create_orga`
  die in der Testumgebung ohne Super-Admin-Rechte bewusst nicht verfügbar sind
- **0 failed**: kein Test schlägt fehl

6 Schlussbemerkungen

Eine fehlende Zeile in einer Basisklasse, auf den ersten Blick ein Entwicklungsfehler wie viele andere. Tatsächlich war die fehlende Mandantenfilterung in `BaseService.get()` kein isoliertes Problem an einem einzelnen Endpunkt, sondern ein systemischer Konstruktionsfehler, der sich auf nahezu alle CRUD-Endpunkte mit organisationsgebundenen Ressourcen auswirkte. Und weil in derselben Methode auch der `deleted_at`-Filter fehlte, liessen sich soft-gelöschte Datensätze per direkter UUID-Abfrage weiterhin lesen. Ein Soft-Delete-Bypass, der ohne diese Analyse vermutlich unentdeckt geblieben wäre.

Die Behebung erfolgte bewusst an der gemeinsamen Wurzel des Problems. Statt einzelne Endpunkte punktuell nachzubessern, wurde die Basisklasse um einen optionalen `organization_id`-Parameter sowie einen konsistenten Soft-Delete-Filter erweitert und anschliessend an allen relevanten Aufrufern korrekt verdrahtet. Dieses Vorgehen reduzierte den Änderungsaufwand pro Endpunkt, verbesserte die Konsistenz der Sicherheitslogik und senkte das Risiko, dass künftig neue Router erneut ohne Mandantenscoping implementiert werden.

Das Testergebnis spricht für sich: Die zuvor als Proof-of-Concept formulierten Angriffe schlagen nach der Anpassung fehl, die neu hinzugefügten Regressionstests decken beide Schwachstellenklassen gezielt ab, und die bestehende Testsuite läuft nach Anpassung der betroffenen Alt-Tests wieder vollständig durch. Der finale Stand mit `187 passed, 2 xfailed` zeigt, dass der Sicherheitsfix nicht nur lokal funktioniert, sondern auch mit dem restlichen Backend vereinbar ist.

Multi-Tenant-Isolation ist kein Komfortmerkmal, sie ist eine Systemeigenschaft. Sobald Ressourcen per UUID adressierbar sind, muss die Autorisierung bei jedem Zugriff explizit an den aktuellen Mandanten gebunden werden. Wird diese Prüfung an einer zentralen Abstraktion vergessen, skaliert die Schwachstelle mit der Wiederverwendung genau so stark wie der Architekturvorteil selbst.

Ein verbleibender Prüfpunkt ausserhalb des bearbeiteten Scopes betrifft Modelle, die nicht über `organization_id`, sondern über benutzerspezifische Beziehungen geschützt werden, insbesondere Benachrichtigungen. Dort liegt keine identische Schwachstelle vor, die Analyse zeigt jedoch, dass derselbe Grundsatz auch für andere Besitzermodelle konsequent weitergeführt werden sollte.

Der Patch erfüllt die Anforderungen an eine nicht-triviale, sicherheitsrelevante Open-Source-Kontribution: Die Schwachstelle wurde technisch analysiert, reproduzierbar nachgewiesen, an der Wurzel behoben und mit einer umfassenden Testsuite abgesichert, eine Grundlage, die auch einem echten Code-Review standzuhalten verspricht.

Quellenverzeichnis

[1] R. Santos. „[SEC-C4] Fix IDOR: add organization_id scoping to BaseService.get_or_404()“. (2026), Adresse: https://github.com/rogeriosantos/dnchub-app/issues/5 (besucht am 28. Apr. 2026).

[2] R. Santos. „[AR-H3] Soft-deleted records accessible via direct ID lookup“. (2026), Adresse: https://github.com/rogeriosantos/dnchub-app/issues/14 (besucht am 28. Apr. 2026).

[3] OWASP Foundation. „Insecure Direct Object Reference Prevention Cheat Sheet“. (2023), Adresse: https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html (besucht am 28. Apr. 2026).

[4] MITRE Corporation. „CWE-639: Authorization Bypass Through User-Controlled Key“. (2024), Adresse: https://cwe.mitre.org/data/definitions/639.html (besucht am 28. Apr. 2026).

[5] FIRST.Org, Inc. „Common Vulnerability Scoring System v3.1: Specification Document“. (2019), Adresse: https://www.first.org/cvss/specification-document (besucht am 28. Apr. 2026).

Eigenständigkeitserklärung

Ich (wir) erkläre(n) hiermit, dass ich (wir) den vorliegenden Leistungsnachweis selber und selbständig verfasst habe(n),

- dass ich (wir) sämtliche nicht von mir (uns) selber stammenden Textstellen und anderen Quellen wie Bilder etc. gemäss gängigen wissenschaftlichen Zitierregeln[1] korrekt zitiert und die verwendeten Quellen klar sichtbar ausgewiesen habe(n);
- dass ich (wir) in einer Fussnote oder einem Hilfsmittelverzeichnis alle verwendeten Hilfsmittel (KI-Assistenzsysteme wie Chatbots[2], Übersetzungs-[3] Paraphrasier-[4] oder Programmierapplikationen[5]) deklariert und ihre Verwendung bei den entsprechenden Textstellen angegeben habe(n);
- dass ich (wir) sämtliche immateriellen Rechte an von mir (uns) allfällig verwendeten Materialien wie Bilder oder Grafiken erworben habe(n) oder dass diese Materialien von mir (uns) selbst erstellt wurde(n);
- dass das Thema, die Arbeit oder Teile davon nicht bei einem Leistungsnachweis eines anderen Moduls verwendet wurden, sofern dies nicht ausdrücklich mit der Dozentin oder dem Dozenten im Voraus vereinbart wurde und in der Arbeit ausgewiesen wird;
- dass ich mir (wir uns) bewusst bin (sind), dass meine (unsere) Arbeit auf Plagiate und auf Drittautorschaft menschlichen oder technischen Ursprungs (Künstliche Intelligenz) überprüft werden kann;
- dass ich mir (wir uns) bewusst bin (sind), dass die Hochschule für Technik FHNW einen Verstoss gegen diese Eigenständigkeitserklärung bzw. die ihr zugrundeliegenden Studierendenpflichten der Studien- und Prüfungsordnung der Hochschule für Technik verfolgt und dass daraus disziplinarische Folgen (Verweis oder Ausschluss aus dem Studiengang) resultieren können.

Windisch, tt. Monat 20jj

Name: Pia Musterfrau

Unterschrift:

Name: Michael Mustermann

Unterschrift:

[1] z.B. APA oder IEEE
[2] z.B. ChatGPT
[3] z.B. Deepl
[4] z.B. Quillbot
[5] z.B. Github Copilot

Declaration of Authenticity

I hereby declare that any individual work / pair work / team work submitted for assessment is entirely the product of my own / my own and my partner's / my own and my team's effort,

- that I/we have correctly cited all text passages that do not originate from me/us, in accordance with standard academic citation rules[6], and that I/we have clearly mentioned all sources used;
- that I/we have declared in footnotes or in an index of auxiliary tools all aids used (AI assistance systems such as chatbots[7], translation[8], paraphrasing[9], or programming applications[10], and indicated their use at the corresponding text passages;
- that I/we have acquired all intangible rights to any materials I/we may have used, such as images or graphics, or that these materials were created by me/us;
- that the topic, the thesis or parts of it have not been used in an assessment of another module, unless this has been expressly agreed with the lecturer in advance and is stated as such;
- that I/we am/are aware that my/our work may be checked for plagiarism and for third-party authorship of human or technical origin (artificial intelligence);
- that I/we am/are aware that the FHNW School of Engineering will pursue a violation of this declaration of authenticity and that disciplinary consequences (reprimand or expulsion from the study program) may result from this.

Windisch, tt. Monat 20jj

Name: Pia Musterfrau

Signature:

Name: Michael Mustermann

Signature:

[6] e.g. APA oder IEEE
[7] e.g., ChatGPT
[8] e.g., DeepL
[9] e.g., Quillbot
[10] e.g., Github Copilot

A Bilddokumentation des Angriffs

Dieser Anhang dokumentiert den Angriff zusätzlich anhand manueller Screenshots. Die Abbildungen dienen nicht als eigenständiger Nachweis der Schwachstelle, sondern als visuelle Ergänzung zur in Kapitel 3 beschriebenen PoC-Analyse und zu den automatisierten Tests aus Kapitel 5.

A.1 UI-basierte Reproduktion

Die UI-Screenshots zeigen den Angriff in einer realistischen Benutzerperspektive: Zunächst wird durch das Opfer ein Fahrzeug angelegt, danach wird dieses Objekt durch einen Benutzer einer fremden Organisation ausserhalb seines erlaubten Mandantenkontexts sichtbar gemacht beziehungsweise trotz Löschung weiterhin angezeigt.

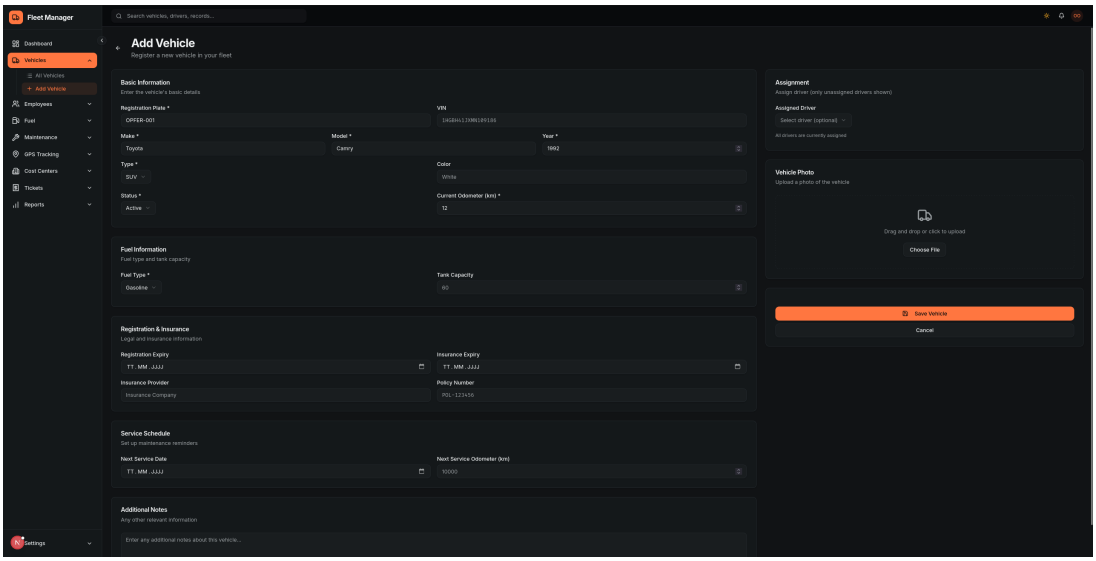


Abbildung A.1: Das Opfer legt im Fleet-Management ein Fahrzeug an.

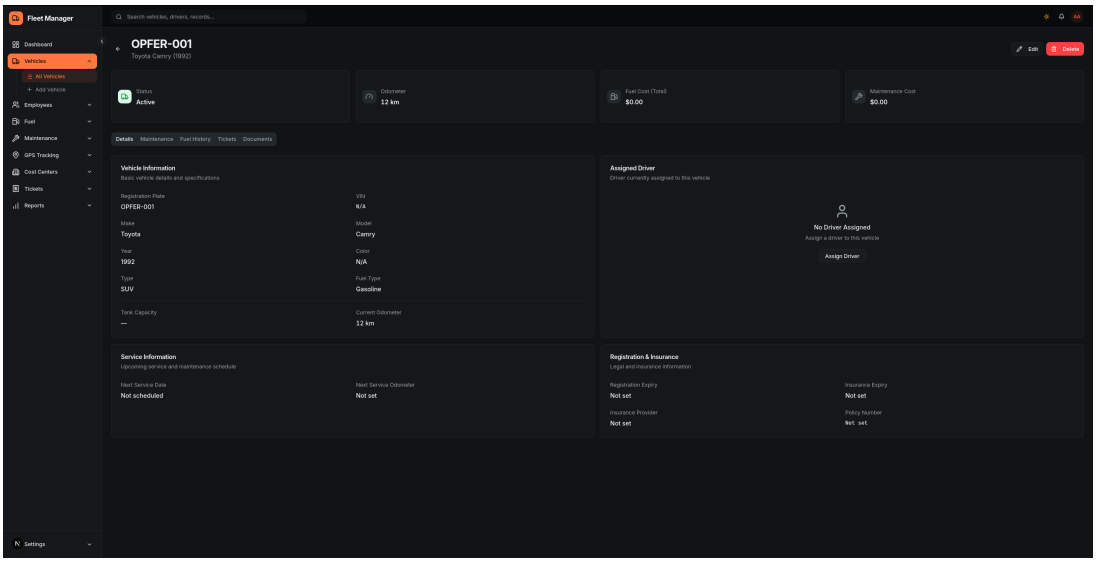


Abbildung A.2: Der Angreifer kann das Fahrzeug des Opfers über die direkte Objekt-ID aufrufen.

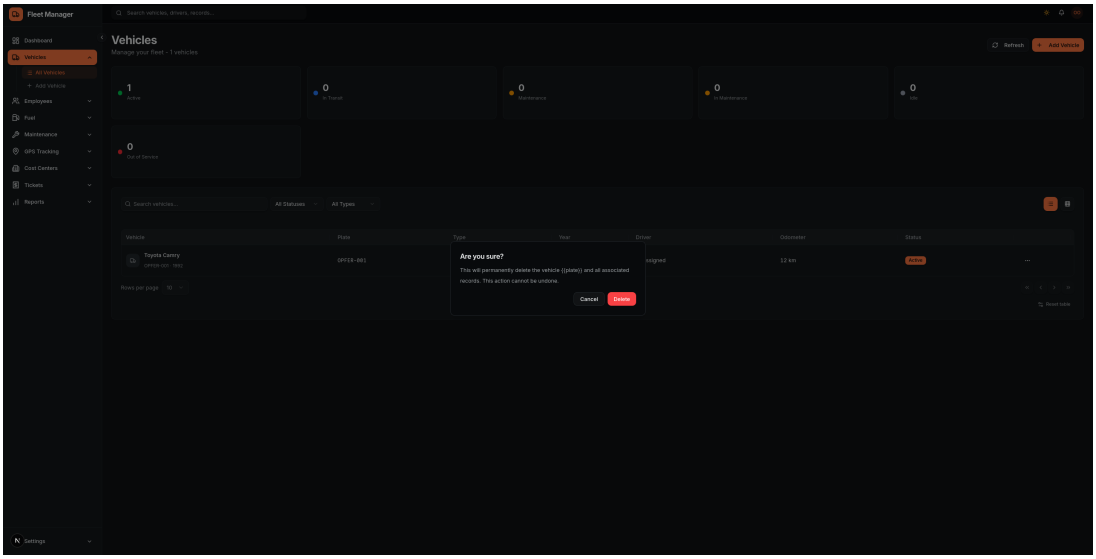


Abbildung A.3: Soft-gelöschte Datensätze bleiben vor dem Patch in der Oberfläche weiterhin erreichbar.

A.2 API-Reproduktion mit curl

Die `curl`-Screenshots zeigen denselben Sachverhalt ohne Frontend. Dadurch ist klar erkennbar, dass es sich nicht um ein Anzeigeproblem des Clients, sondern um einen serverseitigen Autorisierungsfehler handelt. Das Opfer erzeugt ein Fahrzeug, der Angreifer nutzt dessen UUID für einen fremden GET- und DELETE-Zugriff, und das Objekt bleibt sogar nach einem Soft-Delete direkt per ID lesbar.

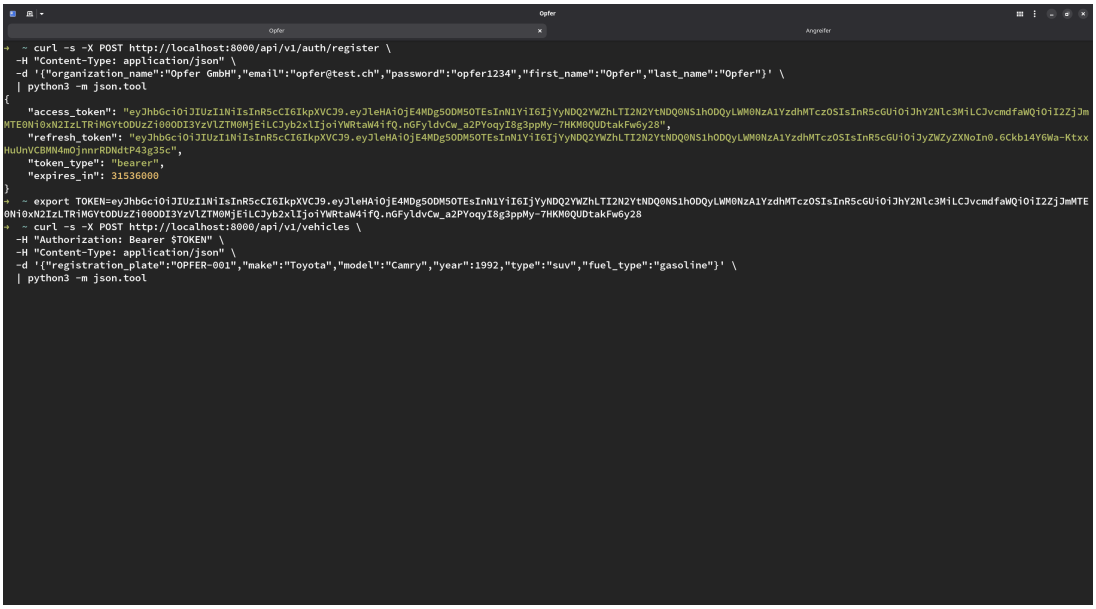


Abbildung A.4: Erzeugung eines Fahrzeugs im Kontext des Opfers per API.

```
Angreifer
Opfer                                                                    Angreifer
→  ~ curl -s -X POST http://localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"organization_name":"Angreifer GMBH","email":"angreifer@test.ch","password":"angreifer","first_name":"Angreifer","last_name":"Angreifer"}' \
  | python3 -m json.tool
{
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODQxMjQsInN1YiI6IjEzMTNjMzY0LWVkODAtNGQ4OS04ZTA4LTkyZDZkNzRkNTczYyIsInR5cGUiOiJhY2Nlc3MiLCJvcmdfaWQiOiJlOTY2
NTljYi01MjcxLTQ1NDUtYTNkMi04MjU5YjM3ZThhM2EiLCJyb2xlIjoiYWRtaW4ifQ.K-bQ2hUZDbmDKtR-LoyaC4acJ2WRMkb3VqKqCU37aIw",
    "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODQxMjQsInN1YiI6IjEzMTNjMzY0LWVkODAtNGQ4OS04ZTA4LTkyZDZkNzRkNTczYyIsInR5cGUiOiJyZWZyZXNoIn0.88-kv5I0gt01gYT
57Psio3ajxnK7hxQIgsp8fUXgrDk",
    "token_type": "bearer",
    "expires_in": 31536000
}
→  ~ export TOKEN=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODQxMjQsInN1YiI6IjEzMTNjMzY0LWVkODAtNGQ4OS04ZTA4LTkyZDZkNzRkNTczYyIsInR5cGUiOiJhY2Nlc3MiLCJvcmdfaWQiOiJlOTY2NTl
jYi01MjcxLTQ1NDUtYTNkMi04MjU5YjM3ZThhM2EiLCJyb2xlIjoiYWRtaW4ifQ.K-bQ2hUZDbmDKtR-LoyaC4acJ2WRMkb3VqKqCU37aIw
→  ~ export VID=2dbd498a-e79b-4f71-8d0a-b1e785d7a718
→  ~
```

Abbildung A.5: Der Angreifer verwendet dieselbe UUID für einen unautorisierten GET-Zugriff.

```
Angreifer
Opfer                                                                    Angreifer
→  ~ export TOKEN=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE4MDg5ODQxMjQsInN1YiI6IjEzMTNjMzY0LWVkODAtNGQ4OS04ZTA4LTkyZDZkNzRkNTczYyIsInR5cGUiOiJhY2Nlc3MiLCJvcmdfaWQiOiJlOTY2NTl
jYi01MjcxLTQ1NDUtYTNkMi04MjU5YjM3ZThhM2EiLCJyb2xlIjoiYWRtaW4ifQ.K-bQ2hUZDbmDKtR-LoyaC4acJ2WRMkb3VqKqCU37aIw
→  ~ export VID=2dbd498a-e79b-4f71-8d0a-b1e785d7a718
→  ~ curl -s -X GET http://localhost:8000/api/v1/vehicles/$VID \
  -H "Authorization: Bearer $TOKEN" \
  | python3 -m json.tool
{
    "deleted_at": null,
    "created_at": "2026-04-29T07:34:48.031902Z",
    "updated_at": "2026-04-29T07:34:48.031902Z",
    "registration_plate": "OPFER-001",
    "vin": null,
    "make": "Toyota",
    "model": "Camry",
    "year": 1992,
    "type": "suv",
    "fuel_type": "gasoline",
    "status": "active",
    "color": null,
    "current_odometer": "0.00",
    "fuel_capacity": null,
    "image": null,
    "insurance_expiry": null,
    "insurance_provider": null,
    "insurance_policy_number": null,
    "registration_expiry": null,
    "last_service_date": null,
    "next_service_date": null,
    "next_service_odometer": null,
    "notes": null,
    "id": "2dbd498a-e79b-4f71-8d0a-b1e785d7a718",
    "organization_id": "6f2f1146-17b3-4b0f-853f-4827c5ee3421",
    "assigned_employee_id": null,
    "cost_center_id": null,
    "average_fuel_efficiency": null,
    "total_fuel_cost": "0.00",
    "total_maintenance_cost": "0.00"
}
→  ~ curl -s -X DELETE http://localhost:8000/api/v1/vehicles/$VID \
  -H "Authorization: Bearer $TOKEN" \
  -w "\nHTTP-Status: %{http_code}\n"

HTTP-Status: 204
→  ~
```

Abbildung A.6: Unautorisierter DELETE-Aufruf des Angreifers gegen das Fahrzeug des Opfers.

```
    "total_fuel_cost": "0.00",
    "total_maintenance_cost": "0.00"
}
→  ~ curl -s -X DELETE http://localhost:8000/api/v1/vehicles/$VID \
  -H "Authorization: Bearer $TOKEN" \
  -w "\nHTTP-Status: %{http_code}\n"

HTTP-Status: 204
→  ~ curl -s -X GET http://localhost:8000/api/v1/vehicles/$VID \
  -H "Authorization: Bearer $TOKEN" \
  | python3 -m json.tool
{
    "deleted_at": "2026-04-29T07:40:23.627920Z",
    "created_at": "2026-04-29T07:34:48.031902Z",
    "updated_at": "2026-04-29T07:40:23.620890Z",
    "registration_plate": "OPFER-001",
    "vin": null,
    "make": "Toyota",
    "model": "Camry",
    "year": 1992,
    "type": "suv",
    "fuel_type": "gasoline",
    "status": "active",
    "color": null,
    "current_odometer": "0.00",
    "fuel_capacity": null,
    "image": null,
    "insurance_expiry": null,
    "insurance_provider": null,
    "insurance_policy_number": null,
    "registration_expiry": null,
    "last_service_date": null,
    "next_service_date": null,
    "next_service_odometer": null,
    "notes": null,
    "id": "2dbd498a-e79b-4f71-8d0a-b1e785d7a718",
    "organization_id": "6f2f1146-17b3-4b0f-853f-4827c5ee3421",
    "assigned_employee_id": null,
    "cost_center_id": null,
    "average_fuel_efficiency": null,
    "total_fuel_cost": "0.00",
    "total_maintenance_cost": "0.00"
}
→  ~
```

Abbildung A.7: Das soft-gelöschte Fahrzeug bleibt vor dem Patch per direkter ID-Abfrage sichtbar.